

**Ecole Supérieure Privée d'Ingénierie et de Technologies –
ESPRIT**

Département : Génie Informatique – Cycle Ingénieur

**Conception et développement d'un
assistant conversationnel IA
e-commerce et de son backoffice
d'administration**

Rapport de stage

présenté en vue de l'obtention du diplôme national d'ingénieur

Élaboré par :

Oussama Wannassi

Encadré par :

Encadrant académique

(ESPRIT) : [Nom à compléter]

Encadrant professionnel

(Prodexo) : [Nom à compléter]

Entreprise d'accueil : Prodexo

Période de stage : du 01/02/2026 au 01/08/2026

Année universitaire 2025 – 2026

Remerciements

Je tiens tout d'abord à exprimer ma sincère reconnaissance à l'ensemble du personnel de **Prodexo** pour l'accueil chaleureux qui m'a été réservé tout au long de ce stage de fin d'études, ainsi que pour la confiance qui m'a été accordée sur un projet à la fois technique et exigeant.

Mes remerciements les plus sincères vont à mon encadrant professionnel, [**Nom à compléter**], pour sa disponibilité, ses conseils avisés et le suivi régulier qu'il/elle a assuré durant les six mois de ce stage, ainsi que pour la liberté et la responsabilité qui m'ont été confiées dans les choix d'architecture et d'implémentation.

Je remercie également mon encadrant académique à l'**ESPRIT**, [**Nom à compléter**], pour son accompagnement méthodologique et ses retours qui ont permis de structurer la démarche de ce travail.

Mes remerciements s'adressent aussi à l'ensemble du corps professoral et administratif de l'ESPRIT, dont la formation en cycle ingénieur a constitué le socle des compétences mobilisées durant ce stage.

Enfin, je remercie les membres du jury d'avoir accepté d'évaluer ce travail, ainsi que toute personne ayant, de près ou de loin, contribué à la réussite de ce stage.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage de fin d'études de six mois effectué au sein de la société **Prodexo**, portant sur la conception et le développement d'une plateforme d'assistant conversationnel intelligent destinée aux sites e-commerce, ainsi que du backoffice permettant à un administrateur de tenant de la configurer et d'en superviser l'activité.

La plateforme repose sur un service central (*ai-core*) développé en **Python/FastAPI**, exposant un chat conversationnel augmenté par recherche vectorielle (*RAG*) sur le catalogue produits, une couche de garde-fous de sécurité (détection d'injection de prompt, masquage de données personnelles), un **moteur de règles métier** permettant de personnaliser le comportement du chat sans redéploiement, ainsi qu'un **agent IA d'administration** à exécution encadrée (workflow d'exécution à blanc, confirmation, double confirmation et annulation) pour les actions à impact sur les données du tenant. Le backoffice, développé en **Laravel / Filament**, consomme exclusivement cette API et fournit un tableau de bord, une interface conversationnelle pour l'agent d'administration, la gestion des règles, ainsi que des pages d'analyse de coût et de comportement client.

Une part significative du travail a consisté à auditer l'existant, à identifier les métriques et fonctionnalités fabriquées ou non connectées à des données réelles, puis à les remplacer par des calculs réels fondés sur les données effectivement collectées par la plateforme (messages, conversations, produits, coupons), en assumant explicitement les limites du système lorsqu'une donnée n'existe pas (absence de table de commandes, par exemple).

Le projet a été mené selon une démarche agile inspirée de **Scrum**, avec une phase d'analyse et de conception suivie de sprints de développement itératifs. Le résultat final est validé par une suite de tests automatisés (tests unitaires et tests d'intégration sur base de données réelle) ainsi que par des vérifications manuelles de l'interface d'administration.

Mots-clés : Intelligence artificielle générative, *RAG (Retrieval-Augmented Generation)*, chatbot e-commerce, FastAPI, moteur de règles métier, agent IA supervisé, Laravel, Filament, architecture multi-tenant, méthodologie Scrum.

Abstract

This report presents the work carried out during a six-month end-of-studies internship at **Prodexo**, focused on the design and development of an AI-powered conversational assistant platform for e-commerce websites, along with the backoffice administration panel used to configure it and monitor its activity.

The platform is built around a central service (*ai-core*) developed with **Python/-FastAPI**, exposing a conversational chat endpoint augmented with vector-based product search (*RAG*), a security guardrails layer (prompt-injection detection, PII masking), a **business rules engine** allowing the chat behavior to be customized without redeployment, and a **supervised admin AI agent** restricted to a predefined set of actions and gated by a dry-run / confirm / rollback workflow for any action affecting tenant data. The backoffice, built with **Laravel / Filament**, is a pure REST client of this API and provides a dashboard, a conversational interface for the admin agent, rules management, and cost/behavior analytics pages.

A significant part of the work consisted of auditing the existing codebase, identifying fabricated or disconnected metrics and features, and replacing them with real computations grounded in the data actually collected by the platform (messages, conversations, products, coupons), while being explicit about the system's limitations where no underlying data exists (e.g., no orders table).

The project followed an agile approach inspired by **Scrum**, with an initial analysis and design phase followed by iterative development sprints. The final result is validated through an automated test suite (unit and integration tests against a real database) as well as manual verification of the admin interface.

Keywords : Generative AI, Retrieval-Augmented Generation (RAG), e-commerce chatbot, FastAPI, business rules engine, supervised AI agent, Laravel, Filament, multi-tenant architecture, Scrum methodology.

Table des matières

Remerciements	ii
Résumé	iii
Abstract	iv
Introduction générale	1
1 Cadre général du projet	2
Introduction	2
1.1 Présentation de l'organisme d'accueil	2
1.2 Contexte du projet	2
1.3 Problématique	3
1.4 Objectifs du projet	3
1.5 Méthodologie de gestion de projet	4
1.5.1 Choix de la méthodologie	4
1.5.2 Découpage temporel du stage	5
1.6 Environnement de travail	5
Conclusion	5
2 Analyse et spécification des besoins	6
Introduction	6
2.1 Identification des acteurs	6
2.2 Besoins fonctionnels	6
2.2.1 Côté client	7
2.2.2 Côté administrateur	7
2.3 Besoins non fonctionnels	7
2.4 Diagramme de cas d'utilisation	8
2.5 Backlog produit	8
Conclusion	9
3 Conception	10
Introduction	10
3.1 Architecture générale	10
3.2 Choix technologiques	11

3.3	Modélisation du domaine	11
3.3.1	Le moteur de règles	11
3.3.2	L'agent IA d'administration	12
3.4	Séquence du chat conversationnel	13
	Conclusion	14
4	Réalisation	16
	Introduction	16
4.1	Sprint 1 – Fondations et infrastructure	16
4.2	Sprint 2 – Moteur de chat IA	17
4.3	Sprint 3 – Sécurité, multi-tenance et supervision	17
4.4	Sprint 4 – Backoffice Laravel/Filament : socle	18
4.5	Sprint 5 – Moteur de règles et agent IA d'administration sécurisé	19
4.6	Sprint 6 – Finalisation du backoffice et documentation	20
	Conclusion	21
5	Tests et validation	22
	Introduction	22
5.1	Stratégie de tests	22
5.1.1	Tests unitaires	22
5.1.2	Tests d'intégration	22
5.1.3	Tests applicatifs (backoffice)	23
5.1.4	Vérification manuelle	23
5.2	Résultats	23
5.3	Exemple : validation du calcul de la demande non satisfaite	24
	Conclusion	24
	Conclusion générale et perspectives	25
	Bibliographie et Netographie	27
A	Extraits de code significatifs	29
A.1	Évaluation pure des règles métier	29
A.2	Classification des actions d'administration par niveau de risque	30
B	Dictionnaire de données	31

Table des figures

2.1	Diagramme de cas d'utilisation de la plateforme	8
3.1	Diagramme de conteneurs (C4, niveau 2) de la plateforme	11
3.2	Diagramme de classes de la plateforme	13
3.3	Séquence : génération de coupons en masse par l'agent d'administration	14
3.4	Séquence complète d'une requête de chat	15
4.1	Diagramme d'activités : évaluation des règles et répartition des actions	20

Liste des tableaux

1.1	Planification générale du stage	5
2.1	Backlog produit (epics, par ordre de priorité)	9
3.1	Principaux choix technologiques	12
5.1	Synthèse de la couverture de tests à la livraison finale	23
B.1	Table tenants	31
B.2	Tables conversations et messages	31
B.3	Table rules	32
B.4	Table coupons	32
B.5	Table products	33

Introduction générale

Le commerce électronique s’est imposé au cours de la dernière décennie comme un canal de vente incontournable, et la pression concurrentielle qui en découle pousse les marchands à offrir une expérience client toujours plus réactive et personnalisée. Dans ce contexte, les assistants conversationnels fondés sur l’intelligence artificielle générative représentent une réponse naturelle : disponibilité continue, réponses instantanées, capacité à orienter le client dans un catalogue produits parfois volumineux. Toutefois, la mise en production de tels assistants soulève des difficultés qui dépassent largement la simple intégration d’un modèle de langage : isolement des données entre marchands (multi-tenance), fiabilité des réponses fournies (absence d’hallucination sur des informations factuelles comme le stock ou le prix), sécurité face aux tentatives de manipulation du modèle, et surtout, capacité pour l’exploitant de la plateforme à comprendre et piloter ce que fait réellement l’assistant.

C’est dans ce cadre que s’inscrit le présent stage de fin d’études, réalisé au sein de la société **Prodexo** sur une période de six mois, du 1^{er} février au 1^{er} août 2026. Le projet confié consistait à faire évoluer une plateforme d’assistant IA e-commerce existante – initialement construite comme un socle technique fonctionnel mais partiellement instrumenté – vers une version exploitable en conditions réelles par un administrateur de boutique, avec deux exigences transversales : que le comportement de l’assistant soit configurable sans intervention technique, et que les données présentées à l’administrateur (tableau de bord, statistiques, recommandations) soient systématiquement réelles et vérifiables, et non de simples valeurs d’exemple.

Ce rapport retrace la démarche suivie, depuis l’analyse du système existant jusqu’à la livraison d’une version consolidée du produit. Il s’organise en cinq chapitres. Le premier chapitre présente le cadre général du stage : l’entreprise d’accueil, le contexte et la problématique du projet, ses objectifs, ainsi que la méthodologie de gestion de projet adoptée. Le deuxième chapitre est consacré à l’analyse et à la spécification des besoins, avec l’identification des acteurs du système et de leurs besoins fonctionnels et non fonctionnels. Le troisième chapitre détaille la conception technique retenue : architecture générale, choix technologiques et modélisation. Le quatrième chapitre décrit la réalisation du projet, sprint par sprint, en mettant en avant les principales fonctionnalités développées. Le cinquième chapitre présente la démarche de tests et de validation mise en place. Le rapport se conclut par un bilan du travail accompli et les perspectives d’évolution envisageables pour la plateforme.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre situe le stage dans son contexte : l'entreprise d'accueil, la problématique à l'origine du projet, les objectifs fixés et la démarche méthodologique retenue pour y répondre.

1.1 Présentation de l'organisme d'accueil

Prodexo est l'entreprise au sein de laquelle ce stage de fin d'études a été réalisé. *[Section à compléter avec la présentation officielle de l'entreprise : secteur d'activité, taille, positionnement, principaux clients ou domaines d'intervention, organigramme du service d'accueil.]*

Le stage s'est déroulé au sein de l'équipe en charge du développement produit, sur un projet interne de plateforme d'assistance IA destinée aux marchands utilisant des solutions e-commerce, en particulier **PrestaShop**.

1.2 Contexte du projet

À l'arrivée sur le projet, une première version de la plateforme existait déjà : un service backend en Python/FastAPI capable de répondre à des messages de chat en s'appuyant sur un modèle de langage, une intégration partielle avec PrestaShop pour la synchronisation du catalogue produits, ainsi qu'une ébauche de backoffice en Laravel/Filament destinée aux administrateurs de boutique.

Cette première version souffrait cependant de deux limites majeures, identifiées dès la phase de prise en main :

- **Un comportement du chat figé dans le code.** Toute personnalisation du comportement de l'assistant (réponses automatiques à certains mots-clés, génération de coupons de fidélisation, politiques de remise) nécessitait une modification

du code source et un redéploiement, ce qui rendait la plateforme peu exploitable par un administrateur non technique.

- **Des données d’administration partiellement fabriquées.** Plusieurs écrans du backoffice – tableau de bord, agent IA d’administration, recommandations – affichaient des valeurs d’exemple codées en dur (chiffre d’affaires fictif, noms de produits génériques) plutôt que des données réellement calculées à partir de l’activité de la plateforme. Ce constat, documenté au fil de l’audit du code existant, a orienté une part importante du projet vers un travail de *fiabilisation* plutôt que de seule addition de fonctionnalités.

1.3 Problématique

La problématique du stage peut se formuler ainsi : *comment permettre à un administrateur de boutique de configurer et de superviser un assistant IA e-commerce en toute confiance, sans dépendre d’une intervention technique pour chaque ajustement, et sans que les informations qui lui sont présentées soient trompeuses ?*

Cette problématique se décline en plusieurs sous-questions techniques traitées tout au long du projet :

- Comment permettre la personnalisation du comportement du chat (réponses automatiques, remises, instructions contextuelles) sans redéploiement, tout en gardant un système simple à auditer ?
- Comment donner à un agent IA la capacité d’exécuter des actions d’administration (génération de coupons en masse, mise à jour de prix) sans lui accorder une exécution libre et potentiellement dangereuse ?
- Comment calculer des indicateurs de pilotage (produits les plus demandés, demande non satisfaite, taux de conversion des coupons) à partir des seules données réellement disponibles, en l’absence – notable – de table de commandes dans le système ?
- Comment garantir, via des tests automatisés reproductibles, que ces garanties (isolation multi-tenant, sécurité de l’agent d’administration, exactitude des métriques) restent vraies dans la durée ?

1.4 Objectifs du projet

Sur la base de cette problématique, les objectifs suivants ont été fixés en début de stage :

1. Concevoir et implémenter un **moteur de règles métier** permettant à un administrateur de définir, via une interface graphique, des conditions (intention

détectée, mots-clés) déclenchant des actions (réponse automatique, instruction injectée dans le contexte du modèle, génération de coupon), interprétées par le chat sans modification de code.

2. Remplacer les données fabriquées de l'agent IA d'administration par des calculs réels, et sécuriser son exécution par un mécanisme d'actions prédéfinies avec confirmation.
3. Concevoir un service d'**analyse de la demande client** (*insights*) fondé exclusivement sur les données réellement collectées par la plateforme.
4. Faire évoluer le backoffice Laravel/Filament pour exposer ces nouvelles capacités : gestion des règles, interface de pilotage de l'agent IA, tableaux de bord à données réelles, suivi des coûts d'utilisation du modèle de langage.
5. Couvrir l'ensemble de ces évolutions par une suite de tests automatisés et documenter l'architecture finale du système.

1.5 Méthodologie de gestion de projet

1.5.1 Choix de la méthodologie

Le projet a été mené selon une démarche agile inspirée du cadre **Scrum** [1]. Ce choix se justifie par la nature du projet : un périmètre fonctionnel amené à être précisé au fil de l'avancement (l'audit de l'existant a lui-même fait émerger une partie du périmètre réel du stage), la nécessité de livrer des incréments testables et démontrables régulièrement, et un stage solo qui bénéficie d'un cadre stagiaire/encadrant proche d'un binôme Product Owner / équipe de développement.

Le déroulement du stage a ainsi suivi les grands principes de Scrum, adaptés à un contexte de stage individuel :

- un **backlog produit** recensant les besoins fonctionnels, priorisé conjointement avec l'encadrant professionnel ;
- un découpage du développement en **sprints** de deux semaines, chacun clos par une démonstration des fonctionnalités livrées ;
- une revue de sprint informelle avec l'encadrant en fin de période, permettant d'ajuster le backlog du sprint suivant ;
- une exigence de *définition of done* stricte pour chaque fonctionnalité : tests automatisés passants (unitaires et, lorsque pertinent, d'intégration sur base de données réelle), vérification manuelle de l'interface concernée, et documentation à jour.

1.5.2 Découpage temporel du stage

Le stage de six mois s’est organisé en deux grandes périodes, détaillées au Chapitre 4 :

- une **phase de cadrage et de conception** (février à avril 2026), consacrée à la prise en main de l’existant, à la formalisation du cahier des charges, à la montée en compétence sur la pile technique (FastAPI, SQLAlchemy asynchrone, Filament) et à la conception de l’architecture cible ;
- une **phase de réalisation** (mai à août 2026), structurée en six sprints de développement itératifs, décrits en détail au chapitre 4.

TABLE 1.1 – Planification générale du stage

Période	Durée	Contenu
Cadrage & conception	Fév. – Avr. 2026	Analyse de l’existant, spécification des besoins, conception de l’architecture
Sprint 1	Mai (S1–S2)	Fondations & infrastructure
Sprint 2	Mai (S3–S4)	Moteur de chat IA (RAG, persistance)
Sprint 3	Juin (S1–S2)	Sécurité, multi-tenant & supervision
Sprint 4	Juin (S3–S4)	Backoffice Laravel/Filament – socle
Sprint 5	Juillet (S1–S2)	Moteur de règles & agent IA d’administration
Sprint 6	Juillet – Août	Finalisation backoffice, analytique réelle, documentation

1.6 Environnement de travail

Le projet a été développé dans un environnement conteneurisé (**Docker** / Docker Compose), avec un dépôt **Git** unique hébergeant les deux composants applicatifs (*ai-core* et *backoffice*) ainsi que l’infrastructure de déploiement. L’intégration continue s’appuie sur **GitHub Actions**, avec analyse de qualité de code (*linting*, formatage) exécutée avant chaque fusion. Le suivi de version a suivi une convention de *commits* typés (**feat**, **fix**, **chore**) facilitant la traçabilité du travail réalisé sprint par sprint.

Conclusion

Ce chapitre a présenté le contexte, la problématique et les objectifs du stage, ainsi que la démarche méthodologique adoptée pour y répondre. Le chapitre suivant détaille l’analyse et la spécification des besoins qui ont découlé de cette problématique.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre présente l'analyse des besoins qui a servi de base à la conception du système : identification des acteurs, formalisation des besoins fonctionnels et non fonctionnels, modélisation des cas d'utilisation, et constitution du backlog produit.

2.1 Identification des acteurs

L'étude de l'existant et les échanges avec l'encadrant professionnel ont permis d'identifier les acteurs suivants, humains et systèmes :

- **Client e-commerce** : le visiteur ou acheteur du site marchand, qui interagit avec l'assistant conversationnel pour rechercher des produits, poser des questions ou demander un code promotionnel.
- **Administrateur** (marchand / tenant) : l'utilisateur du backoffice, responsable de la configuration du comportement de l'assistant, du suivi de son activité et du pilotage des actions d'administration.
- **Agent IA d'administration** : composant système déclenché par l'administrateur, exécutant un ensemble d'actions prédéfinies (analytique, génération de coupons, stratégie marketing) sous supervision.
- **PrestaShop** (acteur externe) : la plateforme e-commerce source du catalogue produits.
- **Fournisseur de modèle de langage** (Groq) : acteur externe sollicité pour la génération de réponses en langage naturel.

2.2 Besoins fonctionnels

Les besoins fonctionnels ont été regroupés par acteur principal.

2.2.1 Côté client

- Poser une question en langage naturel et obtenir une réponse contextualisée par le catalogue produits du tenant.
- Rechercher un produit et obtenir des recommandations pertinentes (produits similaires, produits tendance).
- Obtenir, dans certains contextes définis par l'administrateur (ex. relance panier, fidélisation), un code promotionnel généré automatiquement.
- Voir sa demande traitée de façon fiable même lorsque le modèle de langage ne peut apporter de réponse pertinente (dégradation contrôlée plutôt qu'échec silencieux).

2.2.2 Côté administrateur

- Consulter un tableau de bord présentant l'activité réelle du chat (volume de conversations, taux de reconnaissance d'intention, coût d'utilisation du modèle de langage, blocages de sécurité).
- Définir des **règles métier** (conditions sur l'intention détectée ou des mots-clés, associées à une action : réponse automatique, instruction injectée, génération de coupon), sans intervention technique.
- Interroger un **agent IA d'administration** en langage naturel ou via une action structurée, pour obtenir des statistiques, déclencher une génération de coupons en masse ou obtenir une suggestion de stratégie marketing.
- Confirmer, ou annuler, toute action d'administration à impact avant son exécution effective.
- Consulter les indicateurs de demande client réelle : produits les plus demandés, recherches infructueuses (signal de lacune du catalogue), répartition des intentions, heures de pointe, taux de conversion des coupons, produits en stock faible.
- Parcourir l'historique des conversations traitées par l'assistant.
- Suivre la consommation et le coût d'utilisation du modèle de langage sur une période donnée.

2.3 Besoins non fonctionnels

- **Isolation multi-tenant** : chaque tenant ne doit avoir accès qu'à ses propres données (conversations, règles, produits) ; cette contrainte doit être appliquée de façon systématique au niveau de la couche d'accès aux données, non laissée à la discrétion de chaque endpoint.

- **Sécurité de l'exécution automatisée** : l'agent IA d'administration ne doit pouvoir exécuter qu'un ensemble fermé d'actions prédéfinies, chacune classée par niveau de risque, avec confirmation obligatoire au-delà d'un certain niveau.
- **Exactitude des données présentées** : toute métrique affichée dans le backoffice doit être calculée à partir de données réellement stockées ; à défaut de donnée source, la fonctionnalité doit l'indiquer explicitement plutôt que d'afficher une valeur inventée.
- **Traçabilité** : toute action d'administration exécutée doit être journalisée et consultable a posteriori.
- **Résilience** : la persistance d'un message, l'application d'une règle ou la génération d'un coupon ne doivent pas faire échouer la réponse au client en cas d'indisponibilité ponctuelle d'un composant secondaire (dégradation *best-effort*).
- **Testabilité** : chaque fonctionnalité livrée doit être couverte par des tests automatisés reproductibles, exécutables en intégration continue.

2.4 Diagramme de cas d'utilisation

Le diagramme de la Fig. 2.1 synthétise les cas d'utilisation identifiés, organisés en trois sous-systèmes : les interactions côté client, la gestion administrative, et les intégrations externes (synchronisation du catalogue, indexation vectorielle). Le cas d'utilisation *Configurer l'agent IA* y est réalisé, dans l'implémentation finale, par le module de gestion des règles métier détaillé au Chapter 3.

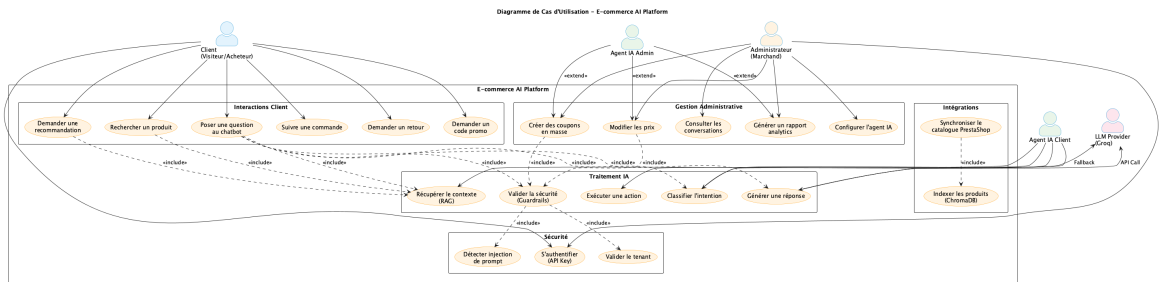


FIGURE 2.1 – Diagramme de cas d'utilisation de la plateforme

2.5 Backlog produit

Le Table 2.1 présente le backlog produit constitué à l'issue de la phase d'analyse, exprimé sous forme d'*epics*. Chaque *epic* a été décomposée en *user stories* au moment de la planification du sprint correspondant (voir Chapter 4).

TABLE 2.1 – Backlog produit (epics, par ordre de priorité)

#	Epic	Valeur métier
1	Fondations techniques (infrastructure, modèles de données, synchronisation catalogue)	Prérequis à toute fonctionnalité
2	Chat conversationnel augmenté (RAG)	Cœur de la proposition de valeur produit
3	Sécurité, multi-tenance et supervision	Condition de mise en production
4	Backoffice – socle (tableau de bord, agent IA – première version)	Autonomie de l’administrateur
5	Moteur de règles métier & agent IA d’administration sécurisé	Personnalisation sans intervention technique
6	Analytique réelle & finalisation backoffice	Confiance dans les données présentées

Conclusion

L’analyse menée dans ce chapitre a permis de traduire la problématique du stage en besoins concrets, priorisés sous forme de backlog. Le chapitre suivant détaille la conception technique retenue pour répondre à ces besoins.

Chapitre 3

Conception

Introduction

Ce chapitre présente les choix de conception retenus pour répondre aux besoins identifiés au chapitre précédent : architecture générale du système, choix technologiques, puis modélisation détaillée (classes, séquences) des mécanismes centraux de la plateforme.

3.1 Architecture générale

Contrairement à une architecture en microservices distincts que l'on aurait pu envisager *a priori* pour un tel système, la plateforme est conçue comme un **monolithe applicatif unique** (*ai-core*) exposant l'ensemble des fonctionnalités métier via une API REST, consommé exclusivement par le backoffice. Ce choix, hérité de la version initiale du projet et confirmé pendant la phase de conception, se justifie par la taille de l'équipe (un stagiaire) et par la simplicité opérationnelle qu'il procure : un seul processus à déployer et à superviser, une seule base de données à administrer, sans les coûts de coordination (cohérence transactionnelle inter-services, observabilité distribuée) qu'imposerait une décomposition en microservices à ce stade de maturité du produit.

Le *ai-core* est le **seul composant autorisé à accéder directement à la base de données PostgreSQL** : cette contrainte est imposée par l'architecture (le backoffice ne détient aucun identifiant de connexion à cette base) et non laissée à la seule discipline des développeurs. Le backoffice Laravel/Filament, à l'inverse, n'a d'autre moyen d'obtenir ou de modifier une donnée que d'appeler l'API REST du *ai-core*.

Comme l'illustre la Fig. 3.1, quatre composants internes structurent le *ai-core* : le **chat** (orchestration garde-fous -> règles -> RAG -> LLM -> persistance), le **moteur de règles**, l'**agent d'administration**, et le service d'**analyse de la demande** (*insights*). Trois dépendances externes complètent l'architecture : **PostgreSQL** (persistance relationnelle, propriété exclusive du *ai-core*), **ChromaDB** (base vectorielle pour la recherche sémantique de produits) et **Redis** (limitation de débit et état des

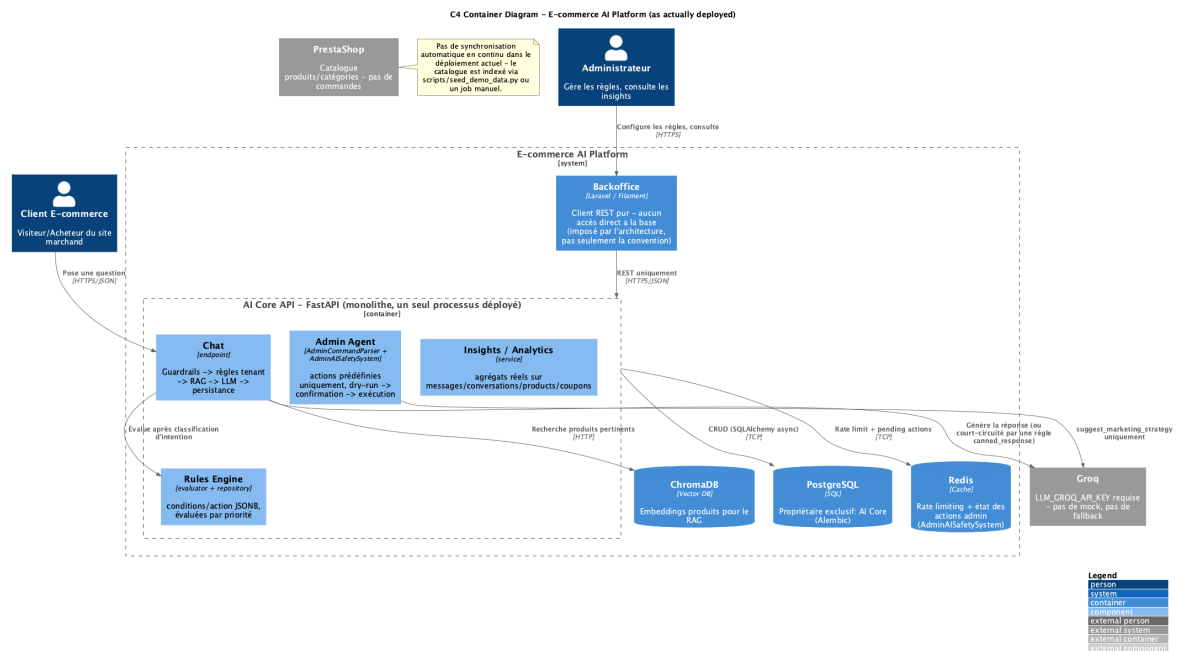


FIGURE 3.1 – Diagramme de conteneurs (C4, niveau 2) de la plateforme actions d’administration en attente de confirmation).

3.2 Choix technologiques

3.3 Modélisation du domaine

Le diagramme de classes de la Fig. 3.2 présente les principales classes du système, regroupées par responsabilité : les entités du domaine persistées en base, le moteur de règles, l’agent d’administration, le service d’analyse de la demande, la couche de sécurité et l’infrastructure (fournisseurs de modèle de langage, recherche vectorielle, unité de travail transactionnelle).

Deux choix de conception méritent d’être détaillés, car ils ont directement découlé des contraintes identifiées au chapitre précédent.

3.3.1 Le moteur de règles

Le moteur de règles repose sur la séparation stricte entre **accès aux données** (RuleRepository, qui charge les règles actives d’un tenant, triées par priorité) et **évaluation** (RuleEvaluator, une fonction pure sans accès base de données ni effet de bord). Une règle est définie par des *conditions* (intention détectée, présence d’au moins un mot-clé parmi une liste, ou présence de tous les mots-clés d’une liste) et une *action* de l’un des trois types suivants : **canned_response** (réponse préétablie, court-circuitant l’appel au modèle de langage), **inject_instruction** (instruction ajoutée au contexte

TABLE 3.1 – Principaux choix technologiques

Composant	Technologie	Justification
API backend	Python 3 / FastAPI	Framework asynchrone performant, typage fort via Pydantic, génération automatique du schéma OpenAPI
Accès aux données	SQLAlchemy 2.0 (async) + Alembic	ORM mature avec support natif de l'asynchrone, migrations versionnées
Base relationnelle	PostgreSQL	Support natif du type JSONB, utilisé pour les conditions/actions des règles et les métadonnées des messages
Recherche sémantique	ChromaDB	Base vectorielle simple à opérer en conteneur, suffisante pour un catalogue produits de taille modérée
Cache / état transitoire	Redis	Nécessaire pour que l'état des actions d'administration en attente de confirmation survive aux redémarrages et soit partagé entre workers
Backoffice	Laravel 13 / Filament 3	Productivité élevée pour construire une interface d'administration (CRUD, tableaux de bord) sans développer un frontend dédié
Interactivité backoffice	Livewire 3	Composants réactifs côté serveur, cohérent avec l'écosystème Filament
Conteneurisation	Docker / Docker Compose	Environnement de développement reproductible, aligné sur le déploiement cible
Intégration continue	GitHub Actions	Exécution automatisée des tests et de l'analyse de qualité à chaque évolution

système envoyé au modèle) ou `generate_coupon` (génération d'un coupon réel après la réponse). Le détail de ce mécanisme est illustré au Chapter 4 par le diagramme d'activités de la Fig. 4.1.

Cette séparation permet de tester exhaustivement la logique de correspondance des règles sans base de données (voir Chapter 5), tout en gardant la persistance et les effets de bord (incrémentement du compteur d'utilisation, génération du coupon) dans la couche appelante.

3.3.2 L'agent IA d'administration

Le choix le plus structurant de ce chapitre concerne l'agent d'administration. Une architecture préexistante mais jamais réellement instanciée proposait un agent générique (`BaseAgent`) reposant sur des services partagés (`LLMGateway`, `RAGService`) eux-

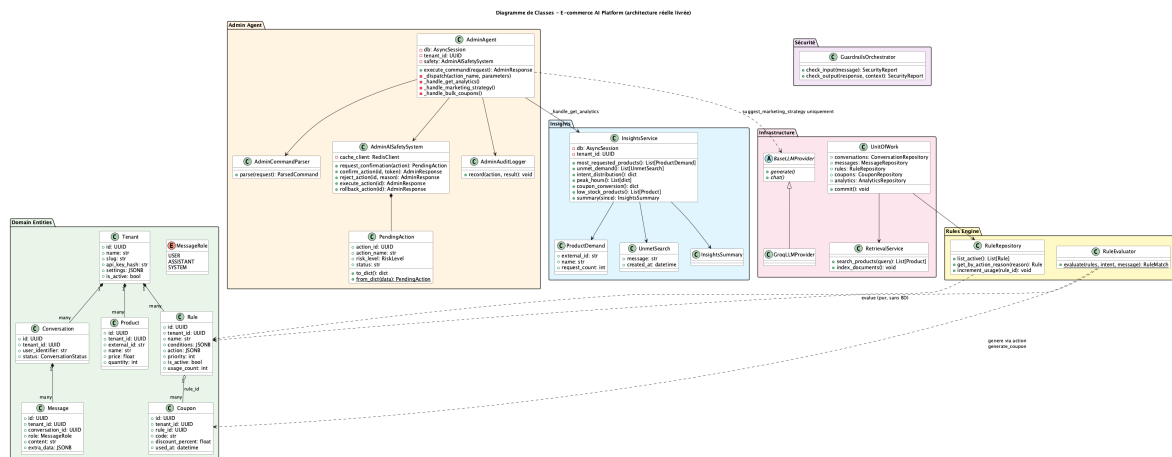


FIGURE 3.2 – Diagramme de classes de la plateforme

mêmes non testés et dépourvus de tout mode simulé, ce qui rendait cette voie coûteuse à finaliser sans bénéfice architectural clair. La décision retenue – après comparaison explicite des deux options avec l’encadrant – a été de **reconstruire l’agent d’administration sur les briques déjà éprouvées du système** : le fournisseur de modèle de langage utilisé par le chat (Groq, sans mode simulé – les tests automatisés s’appuient sur un double de test isolé, jamais atteignable en développement ou en production), le service d’analyse de la demande, et un système de sécurité dédié.

Ce système de sécurité, `AdminAISafetySystem`, restreint l’agent à un ensemble fermé d’actions prédéfinies (`ACTION_DEFINITIONS`), chacune associée à un niveau de risque (faible, moyen, élevé, critique) déterminant le circuit de validation requis : simple confirmation pour un risque moyen, double confirmation avec délai obligatoire pour un risque élevé, mise en file d’attente d’approbation humaine pour un risque critique. Ce workflow – exécution à blanc (*dry-run*), confirmation, exécution, annulation – est illustré en détail par le diagramme de séquence de la Fig. 3.3, pris sur l’exemple concret de la génération de coupons en masse.

Point de conception notable : l'état des actions en attente de confirmation devait initialement être porté par Redis, mais l'implémentation héritée le maintenait en réalité dans un simple dictionnaire Python en mémoire – ce qui aurait rendu toute confirmation invalide après un redémarrage du service, ou incohérente entre plusieurs *workers* applicatifs. Ce point a été corrigé pendant la phase de réalisation (voir [Chapter 4](#), [Sprint 5](#)) en implémentant le stockage réel dans Redis, avec dégradation explicite en mémoire lorsqu'aucun client de cache n'est fourni (utile pour les tests unitaires existants).

3.4 Séquence du chat conversationnel

La Fig. 3.4 détaille le cheminement complet d'un message client, de sa réception à la persistance de la réponse : validation par les garde-fous de sécurité en entrée, clas-

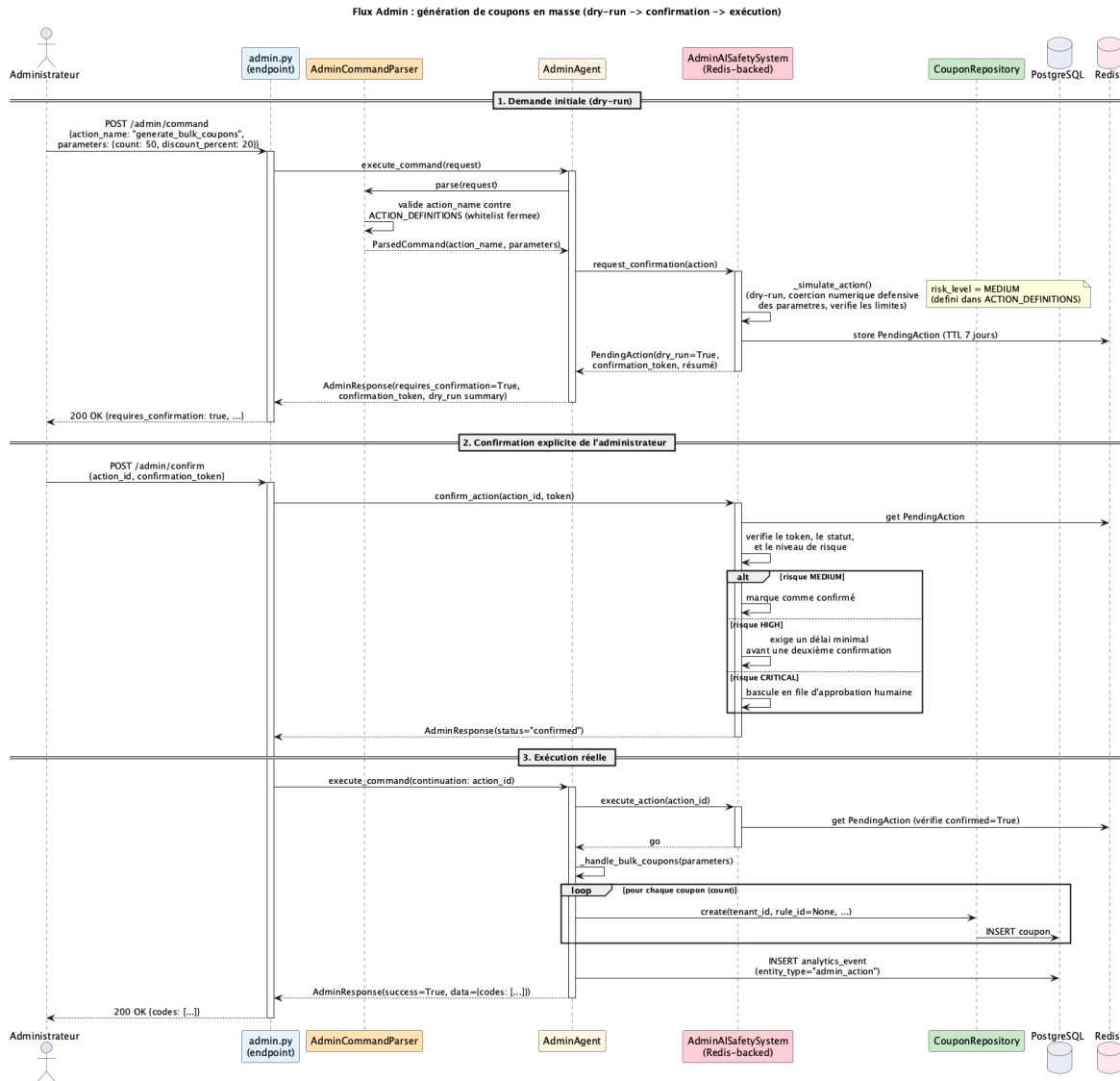


FIGURE 3.3 – Séquence : génération de coupons en masse par l’agent d’administration

sification de l’intention, évaluation des règles métier du tenant, puis – selon le résultat – court-circuit par une réponse préétablie, ou poursuite vers la recherche vectorielle de produits (RAG), l’appel au modèle de langage et la validation des garde-fous en sortie.

Conclusion

Ce chapitre a présenté l’architecture retenue pour la plateforme ainsi que la modélisation des mécanismes centraux du système. Le chapitre suivant décrit la réalisation de ces éléments, sprint par sprint.

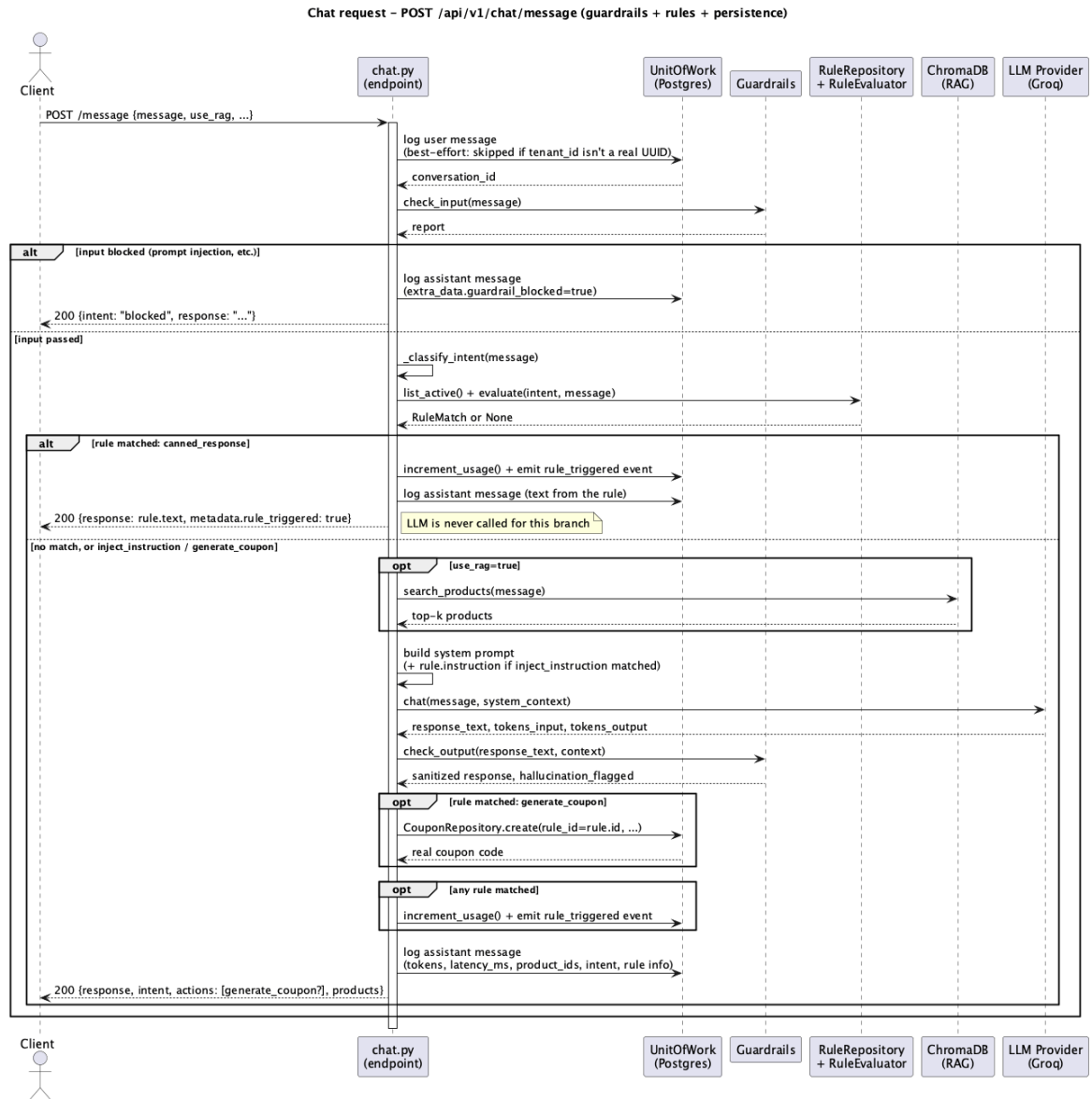


FIGURE 3.4 – Séquence complète d'une requête de chat

Chapitre 4

Réalisation

Introduction

Ce chapitre détaille la phase de réalisation du projet, organisée en six sprints de deux semaines (cf. Table 1.1). Pour chaque sprint sont présentés les objectifs, le contenu livré et, le cas échéant, les difficultés rencontrées.

4.1 Sprint 1 – Fondations et infrastructure

Objectifs

Mettre en place l’environnement de développement conteneurisé, le socle de données du domaine, et la connexion en lecture au catalogue PrestaShop.

Réalisation

Le sprint a débuté par la mise en place de l’**infrastructure Docker Compose** regroupant l’ensemble des services de la plateforme (*ai-core*, backoffice, PostgreSQL, Redis, ChromaDB, Prometheus, Grafana), ainsi que le pipeline d’**intégration continue** sur GitHub Actions et les **migrations Alembic** versionnant le schéma de base de données.

Les **modèles de données du domaine** ont été implémentés : **Tenant** (isolation multi-tenant), **Conversation** et **Message** (historique du chat), **Product** (catalogue) et **Coupon**. La configuration PostgreSQL a été durcie (variables d’environnement sécurisées, taille de pool de connexions configurable, *healthchecks* de conteneur).

Enfin, un **client PrestaShop asynchrone** a été développé (couvert par 33 tests unitaires), ainsi qu’un **service de synchronisation de catalogue** multi-tenant réalisant un *upsert* en masse des produits, avec indexation vers ChromaDB optimisée par pagination en flux continu et évitement des ré-indexations inutiles par comparaison de hachage de contenu.

4.2 Sprint 2 – Moteur de chat IA

Objectifs

Livrer le cœur fonctionnel du chat conversationnel : recherche augmentée par récupération de contexte (RAG) et orchestration de bout en bout de la réponse.

Réalisation

La **recherche vectorielle de produits** (RAG) a été intégrée à l'endpoint de chat, avec prise en charge du contexte multi-tenant et repli explicite (réponse générique, sans blocage) lorsque la recherche ne retourne aucun résultat pertinent. Le patron **Unit of Work** a été mis en place pour garantir qu'une conversation entière (persistance du message utilisateur, de la réponse, des métadonnées associées) s'exécute dans une transaction cohérente. Le service d'orchestration du chat (**ChatService**) a été finalisé avec une logique d'idempotence permettant de court-circuiter le traitement d'une requête déjà traitée, ainsi que des garanties de sécurité vis-à-vis des accès concurrents sur une même conversation.

4.3 Sprint 3 – Sécurité, multi-tenance et supervision

Objectifs

Fiabiliser la plateforme avant montée en charge réelle : sécurité applicative, isolation multi-tenant, observabilité, et correction des dysfonctionnements découverts lors des tests d'intégration approfondis.

Réalisation

Ce sprint a été marqué par la découverte, via des tests d'intégration plus poussés que ceux existants jusqu'alors, de plusieurs écarts significatifs entre le comportement attendu du système et son comportement réel :

- les **garde-fous de sécurité** (**GuardrailsOrchestrator**, détection d'injection de prompt, masquage de données personnelles) existaient dans le code mais n'étaient **pas effectivement appelés** par l'endpoint de chat en production – ils ont été câblés dans le chemin réel de traitement d'un message ;
- la **limitation de débit** (*rate limiting*) était silencieusement désactivée sur l'ensemble des routes ;

- l'**authentification par clé API** des tenants ne fonctionnait pas de bout en bout et a dû être corrigée pour être réellement opérante ;
- une **faille de contournement d'authentification** a été identifiée dans le middleware de résolution du tenant et corrigée en priorité ;
- la **persistance des conversations et des messages**, pourtant présente dans le code depuis le sprint 2, ne s'exécutait en réalité jamais dans certains chemins – aucune conversation n'était donc durablement enregistrée. Le défaut a été isolé et corrigé, de même que plusieurs points d'entrée laissés à l'état d'ébauche (historique de conversation, retour utilisateur, clôture de conversation).

Ces corrections, bien que non prévues telles quelles dans le backlog initial, ont été traitées en priorité dès leur découverte : elles conditionnaient la fiabilité de toute fonctionnalité construite par la suite. Le sprint s'est conclu par la mise en place du **supervisionnement applicatif** (métriques Prometheus, tableaux de bord Grafana), la correction de plusieurs métriques instrumentées mais jamais alimentées (compteurs de coût et de tokens du modèle de langage, score de confiance des réponses, panneaux d'alerte affichant systématiquement l'absence de donnée), ainsi que par la résolution de treize vulnérabilités logicielles (CVE) détectées par l'analyse de dépendances, et la mise en place d'une analyse de qualité de code continue (SonarCloud).

4.4 Sprint 4 – Backoffice Laravel/Filament : socle

Objectifs

Construire le socle du backoffice d'administration : structure de l'application Filament, client REST vers l'API *ai-core*, premières pages d'administration.

Réalisation

L'application **Laravel 13 / Filament 3** a été initialisée, avec un client REST dédié (**AiCoreClient**) constituant le **seul point d'accès** du backoffice vers l'API *ai-core* – aucune autre classe de l'application n'effectue d'appel HTTP direct vers ce service, ce qui centralise la gestion des erreurs réseau et la dégradation en cas d'indisponibilité. Un premier tableau de bord et une première version de la page *Agent IA Admin* ont été livrés, ainsi qu'une page de paramétrage du tenant.

L'intégration avec PrestaShop a été complétée côté marchand par un **module réel** embarquant un widget de chat IA directement sur le site marchand. Plusieurs anomalies bloquantes ont été corrigées pendant ce sprint, notamment un **scénario de verrouillage total et systématique** du panneau d'administration (l'administrateur perdait l'accès au backoffice dans une situation reproductible à 100 %), des erreurs

HTTP 403 intermittentes liées à la configuration de session, ainsi que l’activation du mode *WAL* de SQLite pour supporter des accès concurrents lors des tests automatisés.

4.5 Sprint 5 – Moteur de règles et agent IA d’administration sécurisé

Objectifs

Livrer les deux fonctionnalités centrales du stage : le moteur de règles métier permettant de configurer le chat sans redéploiement, et la refonte de l’agent IA d’administration sur des bases réelles et sécurisées.

Réalisation

Moteur de règles. Le modèle `Rule`, préexistant dans le schéma de base de données mais jamais exploité, a été relié au flux de traitement du chat. L’évaluation des règles (`RuleEvaluator`) a été implémentée comme une fonction pure, testée unitairement de façon exhaustive (voir Chapter 5), puis intégrée dans l’endpoint de chat juste après la classification d’intention : en cas de correspondance, l’action associée (réponse pré-établie, instruction injectée, génération de coupon) est appliquée selon le mécanisme détaillé à la Fig. 4.1. Une interface complète de gestion des règles (création, modification, suppression) a été exposée côté API puis côté backoffice. La politique de remise historiquement codée en dur dans le service de coupons a été repliée dans ce même mécanisme de règles, avec conservation d’une valeur par défaut en l’absence de règle applicable.

Agent IA d’administration. Conformément au choix de conception exposé au Chapter 3, l’agent d’administration a été entièrement reconstruit. Un analyseur de commandes (`AdminCommandParser`) restreint désormais l’agent à un ensemble fermé d’actions prédéfinies, qu’elles soient exprimées de façon structurée ou en langage naturel (classification par mots-clés). Le système de sécurité (`AdminAISafetySystem`) a été effectivement adossé à Redis – corrigeant un écart entre la documentation du composant, qui annonçait un état persistant, et son implémentation réelle, qui le maintenait jusque-là en mémoire locale du processus.

Les données présentées par l’agent (statistiques, produits en demande, demande non satisfaite) ont été entièrement reconstruites à partir du nouveau service `InsightsService`, remplaçant des valeurs d’exemple fixes (chiffre d’affaires fictif, noms de produits génériques) par des agrégats calculés sur les messages, conversations, produits et coupons réellement enregistrés. Un défaut a été détecté lors des essais manuels de l’interface

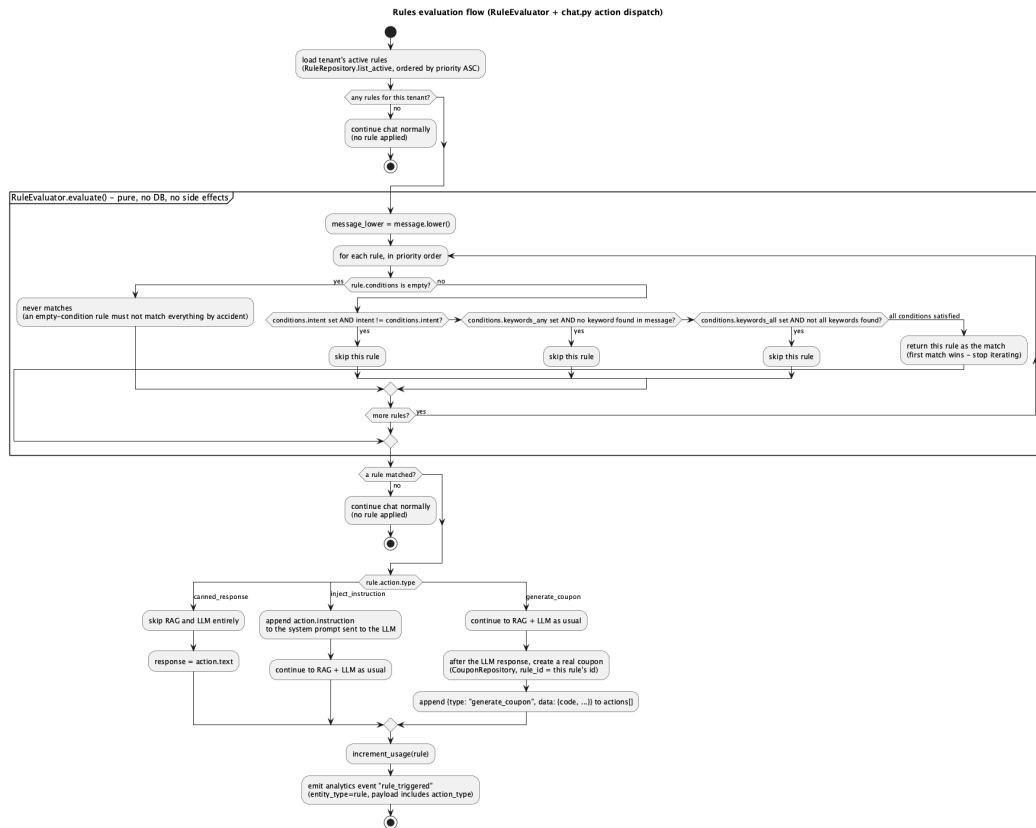


FIGURE 4.1 – Diagramme d’activités : évaluation des règles et répartition des actions

d’administration reconstruite : les paramètres saisis via le formulaire du backoffice étant systématiquement transmis sous forme de chaînes de caractères, une comparaison numérique effectuée lors de la simulation d’une action (*dry-run*) provoquait une erreur de type non gérée. Ce défaut a été corrigé par une coercition défensive des paramètres numériques, et une régression a été ajoutée à la suite de tests pour empêcher sa réintroduction.

4.6 Sprint 6 – Finalisation du backoffice et documentation

Objectifs

Achever l’exposition, côté backoffice, des fonctionnalités livrées au sprint précédent, et mettre à jour la documentation du projet en fin de stage.

Réalisation

Le tableau de bord du backoffice a été doté de **graphiques réels** (Chart.js), alimentés par un nouvel endpoint d’API dédié aux séries temporelles, remplaçant les

graphiques statiques initiaux. Une page *Coûts & Messages* a été ajoutée pour le suivi de la consommation du modèle de langage (tokens, coût, latence moyenne, messages les plus coûteux), ainsi qu’un navigateur de conversations permettant de consulter l’historique complet des échanges d’un tenant, et une page *Insights* restituant l’ensemble des indicateurs de demande calculés par le service dédié.

L’interface de l’agent IA d’administration, jugée initialement trop rudimentaire (un simple formulaire de sélection d’action), a été retravaillée en une véritable **interface de type conversation**, conservant l’historique des échanges dans la session et présentant chaque résultat (statistiques, propositions de stratégie marketing, codes de coupons générés) sous une forme visuelle adaptée à sa nature plutôt qu’en JSON brut. Un défaut d’expérience utilisateur a également été corrigé sur les graphiques de répartition des intentions : en l’absence de toute donnée sur la période sélectionnée, un graphique vide s’affichait sans distinction visuelle avec un graphique en échec technique ; un état vide explicite a été introduit.

Le sprint s’est conclu par la consolidation de la documentation du projet : réécriture du README, mise à jour des diagrammes d’architecture pour refléter le système effectivement livré plutôt que sa conception initiale, et export automatisé du schéma OpenAPI de l’API depuis l’application elle-même, garantissant sa cohérence avec le code réel.

Conclusion

Ce chapitre a détaillé, sprint par sprint, la réalisation du projet. Le chapitre suivant présente la démarche de tests et de validation mise en œuvre pour garantir la fiabilité de l’ensemble.

Chapitre 5

Tests et validation

Introduction

Ce chapitre présente la démarche de tests mise en œuvre tout au long du projet, ainsi que les résultats obtenus sur la version finale livrée. Conformément à la *definition of done* adoptée dès le premier sprint (Chapter 1), aucune fonctionnalité n’a été considérée terminée sans couverture de test correspondante.

5.1 Stratégie de tests

Trois niveaux de tests ont été mis en œuvre, chacun avec un objectif distinct.

5.1.1 Tests unitaires

Les tests unitaires couvrent la logique pure, sans dépendance externe (base de données, cache, réseau). C’est en particulier le cas de l’évaluateur de règles (`RuleEvaluator`), testé à l’aide d’objets de substitution légers plutôt que de véritables enregistrements en base, ce qui permet une exécution rapide et déterministe. Treize scénarios y sont couverts : correspondance sur l’intention, sur un mot-clé parmi plusieurs, sur l’ensemble de mots-clés requis, absence de correspondance, priorité entre règles concurrentes, et le cas limite d’une règle sans aucune condition (qui, par construction, ne doit jamais correspondre par accident).

5.1.2 Tests d’intégration

Les tests d’intégration s’exécutent contre une véritable instance PostgreSQL (ignorés automatiquement si aucune instance n’est joignable, pour ne pas bloquer un environnement de développement incomplet) et valident des scénarios de bout en bout via de véritables appels HTTP à l’application. C’est à ce niveau qu’ont été validés, par exemple : la création réelle d’un coupon en base par une règle de type `generate_coupon`, l’incrémentation du compteur d’utilisation d’une règle après plusieurs déclenchements, le court-circuit effectif du modèle de langage par une réponse

préétablie (vérifié à l’aide d’un fournisseur de modèle de langage instrumenté, garantissant qu’il n’est jamais sollicité dans ce cas), les trois paliers de risque du système de sécurité de l’agent d’administration, ou encore l’exactitude des indicateurs du service d’analyse de la demande sur un jeu de données de test dont le résultat attendu est calculé manuellement puis comparé à la valeur produite par le système.

5.1.3 Tests applicatifs (backoffice)

Côté backoffice, les tests s’appuient sur le simulateur HTTP de Laravel pour isoler chaque page Filament de l’API réelle (réponses simulées), et vérifient à la fois le comportement nominal et la dégradation gracieuse en cas d’indisponibilité du *ai-core* (page fonctionnelle affichant un message d’erreur plutôt qu’une erreur fatale).

5.1.4 Vérification manuelle

Les tests automatisés ne suffisant pas à eux seuls à garantir la qualité perçue d’une interface, chaque page du backoffice a par ailleurs été vérifiée manuellement en conditions réelles : démarrage de la pile applicative complète, injection de données de test représentatives via l’API, puis inspection visuelle du rendu. Cette étape a permis de détecter des défauts qu’aucun test automatisé n’aurait révélés : une erreur de type sur un paramètre transmis sous forme de chaîne par un formulaire (Chapter 4, Sprint 5), ou l’absence de distinction visuelle entre un graphique légitimement vide et un graphique en échec (Sprint 6).

5.2 Résultats

TABLE 5.1 – Synthèse de la couverture de tests à la livraison finale

Composant	Tests passants	Outils
<i>ai-core</i> (Python)	563	pytest (unitaires + intégration)
Backoffice (PHP)	36	PHPUnit / Livewire testing

Au-delà des tests fonctionnels, chaque livraison a été systématiquement validée par les outils d’analyse statique du projet : `ruff check` et `ruff format --check` côté Python, `pint -test` (respect des conventions de style Laravel) côté PHP – les deux exécutés sans erreur sur la version finale.

5.3 Exemple : validation du calcul de la demande non satisfaite

Un exemple illustre la rigueur recherchée dans les tests d'intégration : le calcul de la *demande non satisfaite* (recherches client n'ayant renvoyé aucun résultat produit pertinent) nécessitait d'associer chaque réponse infructueuse de l'assistant au message utilisateur qui l'avait précédée. Une première implémentation, fondée sur un rapprochement par proximité temporelle, s'est révélée non déterministe en cas de messages horodatés de façon identique lors des tests. Le calcul a été repensé pour s'appuyer sur l'invariant réel du système – chaque message utilisateur est immédiatement suivi de la réponse de l'assistant dans le même flux de traitement – en associant chaque paire par rang au sein de la conversation plutôt que par horodatage, rendant le résultat déterministe et vérifiable par un test d'intégration exact.

Conclusion

La démarche de tests mise en place a permis de livrer une version du système dont le comportement – y compris les cas limites et les mécanismes de sécurité – est vérifié de façon reproductible, condition jugée nécessaire au regard de la problématique de confiance posée en introduction de ce rapport.

Conclusion générale et perspectives

Ce stage de fin d'études, réalisé au sein de Prodexo sur une période de six mois, avait pour objectif de faire évoluer une plateforme d'assistant IA e-commerce d'un socle technique fonctionnel vers un produit réellement exploitable par un administrateur de boutique. Cet objectif se déclinait en deux exigences complémentaires : rendre le comportement de l'assistant configurable sans intervention technique, et garantir que les informations présentées à l'administrateur reflètent fidèlement l'activité réelle du système.

Les travaux menés ont permis de livrer un **moteur de règles métier** intégré au flux de traitement du chat, un **agent IA d'administration** restreint à un ensemble d'actions prédéfinies et protégé par un workflow de confirmation adapté au niveau de risque de chaque action, un **service d'analyse de la demande client** fondé exclusivement sur des données réellement collectées, ainsi qu'un **backoffice** complet exposant l'ensemble de ces capacités. Une part substantielle de ce travail a consisté à auditer méthodiquement l'existant pour distinguer ce qui fonctionnait réellement de ce qui n'était qu'apparence – démarche qui s'est révélée aussi formatrice que l'écriture de code neuf, et qui a mis en lumière plusieurs anomalies significatives (garde-fous de sécurité non câblés, persistance de conversations défaillante, données d'administration fabriquées) corrigées au fil des sprints.

Sur le plan personnel, ce stage a été l'occasion d'approfondir des compétences en développement backend asynchrone (FastAPI, SQLAlchemy), en conception d'API, en sécurisation de systèmes automatisés à effet réel, ainsi qu'en développement d'interfaces d'administration avec l'écosystème Laravel/Filament. Il a également permis de pratiquer une démarche de gestion de projet agile de bout en bout, de la constitution d'un backlog à la livraison itérative de sprints testés et documentés.

Plusieurs pistes d'évolution se dégagent naturellement de ce travail pour la suite du projet :

- l'intégration d'une véritable table de commandes, aujourd'hui absente du système, qui permettrait d'enrichir significativement les indicateurs d'analyse (taux de conversion réel, chiffre d'affaires attribuable au chat) au-delà des seuls signaux de demande actuellement disponibles ;
- l'extension du moteur de règles à des conditions plus riches (segmentation client, historique d'achat) une fois cette donnée disponible ;
- la mise en place d'un parcours d'inscription et de facturation self-service complet

pour les tenants, actuellement limité à un mode de développement ;

- l’industrialisation du déploiement en production, au-delà de l’environnement Docker Compose utilisé en développement.

Ce travail constitue une base solide et honnêtement documentée sur laquelle ces évolutions pourront s’appuyer.

Bibliographie et Netographie

Bibliographie

- [1] Schwaber, K. et Sutherland, J. *The Scrum Guide*. 2020. <https://scrumguides.org/>
- [2] *FastAPI Documentation*. 2026. <https://fastapi.tiangolo.com/>
- [3] *SQLAlchemy 2.0 Documentation*. 2026. <https://docs.sqlalchemy.org/en/20/>
- [4] *Filament – Rapid Laravel Development Framework*. 2026. <https://filamentphp.com/docs>
- [5] *Laravel Livewire Documentation*. 2026. <https://livewire.laravel.com/docs>
- [6] *Chroma – The AI-native Open-source Embedding Database*. 2026. <https://docs.trychroma.com/>
- [7] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. 2025. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [8] Brown, S. *The C4 model for visualising software architecture*. 2026. <https://c4model.com/>
- [9] *PrestaShop Webservice API Documentation*. 2026. <https://devdocs.prestashop-project.org/8/webservice/>
- [10] *Redis Documentation*. 2026. <https://redis.io/docs/>

Annexe A

Extraits de code significatifs

Cette annexe présente deux extraits représentatifs, tirés tels quels du code source livré, illustrant les mécanismes centraux décrits au Chapter 3.

A.1 Évaluation pure des règles métier

L'extrait ci-dessous (`app/services/rules/evaluator.py`) montre l'implémentation du `RuleEvaluator` : une fonction sans accès à la base de données, ce qui la rend testable de façon exhaustive et rapide (voir Chapter 5).

```
1 @staticmethod
2 def _matches_conditions(conditions, intent, message_lower):
3     if not conditions:
4         return False
5
6     known_keys = ("intent", "keywords_any", "keywords_all")
7     if not any(key in conditions for key in known_keys):
8         return False
9
10    if "intent" in conditions and conditions["intent"] != intent:
11        return False
12
13    if "keywords_any" in conditions:
14        keywords = [kw.lower() for kw in conditions["keywords_any"]]
15        if not any(kw in message_lower for kw in keywords):
16            return False
17
18    if "keywords_all" in conditions:
19        keywords = [kw.lower() for kw in conditions["keywords_all"]]
20        if not all(kw in message_lower for kw in keywords):
21            return False
22
23    return True
24
25 def evaluate(self, rules, intent, message):
26     """Retourne la premiere regle qui matche, ou None."""
27     message_lower = message.lower()
```

```

28     for rule in rules:
29         if self._matches_conditions(rule.conditions or {}, intent,
30                                     message_lower):
31             return RuleMatch(rule=rule, action=rule.action or {})
32     return None

```

Listing A.1 – Correspondance des conditions d’une règle

Le choix explicite qu’une règle sans aucune des trois clés de condition connues ne corresponde jamais (plutôt que de matcher par défaut) évite qu’une règle mal configurée n’intercepte silencieusement l’intégralité du trafic d’un tenant.

A.2 Classification des actions d’administration par niveau de risque

L’extrait ci-dessous (`app/core/security/admin_safety.py`) montre un sous-ensemble du registre `ACTION_DEFINITIONS`, qui constitue la liste fermée des actions que l’agent d’administration est autorisé à exécuter. Chaque action y est associée à un niveau de risque déterminant le circuit d’approbation requis (cf. Fig. 3.3).

```

1 ACTION_DEFINITIONS: Dict[str, ActionDefinition] = {
2     # RISQUE FAIBLE - execution automatique
3     "get_analytics": ActionDefinition(
4         name="get_analytics",
5         description="Recupere les analytics (lecture seule)",
6         risk_level=RiskLevel.LOW,
7         approval_type=ApprovalType.NONE,
8         supports_rollback=False,
9     ),
10    # RISQUE ELEVE - double confirmation + delai
11    "generate_bulk_coupons": ActionDefinition(
12        name="generate_bulk_coupons",
13        description="Genere des coupons en masse",
14        risk_level=RiskLevel.HIGH,
15        approval_type=ApprovalType.DOUBLE,
16        requires_reason=True,
17        cooldown_seconds=3600, # 1h entre deux generations en masse
18    ),
19    ...
20 }

```

Listing A.2 – Extrait du registre des actions administratives autorisées

Toute action absente de ce registre est rejetée par l’`AdminCommandParser` avant même d’atteindre la logique d’exécution – l’agent ne dispose d’aucune capacité d’exécution libre.

Annexe B

Dictionnaire de données

Cette annexe présente les principales tables du schéma relationnel, propriété exclusive du service *ai-core* (Chapter 3). Les colonnes techniques communes (identifiant primaire [Universally Unique Identifier \(UUID\)](#), horodatages de création/mise à jour) sont omises par souci de lisibilité lorsqu'elles ne portent pas d'information métier spécifique.

TABLE B.1 – Table `tenants`

Colonne	Description
<code>name, slug</code>	Identité du tenant (marchand)
<code>api_key_hash</code>	Hachage de la clé d'API utilisée pour l'authentification serveur-à-serveur
<code>email, is_verified, verification_token_hash</code>	Inscription self-service avec vérification par courriel
<code>settings</code>	Paramètres libres du tenant (JSON Binary (type de données PostgreSQL) (JSONB))
<code>is_active</code>	Active ou désactive le tenant sans suppression de ses données

TABLE B.2 – Tables `conversations` et `messages`

Colonne	Description
<code>conversations.tenant_id</code>	Clé étrangère vers le tenant propriétaire (isolation multi-tenant)
<code>conversations.user_id</code>	Identifiant du client côté site marchand
<code>conversations.status</code>	État de la conversation (ouverte, fermée)
<code>messages.role</code>	USER / ASSISTANT / SYSTEM
<code>messages.content</code>	Contenu textuel du message
<code>messages.idempotency_key</code>	Empêche le double traitement d'une requête réémise
<code>messages.extra_data</code>	Métadonnées structurées (JSONB) : intention détectée, produits retournés, règle éventuellement déclenchée, usage RAG
<code>messages.tokens_input</code>	Montre le coût et de performance de l'appel au Large Language Model (grand modèle de langage) (LLM)
<code>latency_ms</code>	

Remarque. Aucune table de commandes n'existe dans ce schéma : le catalogue synchronisé depuis PrestaShop se limite aux produits et catégories. Toute métrique de

TABLE B.3 – Table rules

Colonne	Description
<code>tenant_id</code>	Isolation multi-tenant
<code>name, description</code>	Libellé de la règle, affiché dans le backoffice
<code>conditions</code>	<code>JSONB</code> – clés supportées : <code>intent</code> , <code>keywords_any</code> , <code>keywords_all</code>
<code>action</code>	<code>JSONB</code> – type (<code>canned_response</code> / <code>inject_instruction</code> / <code>generate_coupon</code>) et paramètres associés
<code>priority</code>	Ordre d'évaluation (croissant) entre règles concurrentes
<code>is_active</code>	Permet de désactiver une règle sans la supprimer
<code>usage_count</code>	Compteur de déclenchements, incrémenté à chaque correspondance

TABLE B.4 – Table coupons

Colonne	Description
<code>tenant_id</code> , <code>conversation_id</code>	Contexte d'émission du coupon
<code>rule_id</code>	Règle à l'origine de la génération, le cas échéant (<code>NULL</code> si généré par l'agent d'administration)
<code>code</code>	Code promotionnel généré
<code>discount_percent</code> / <code>discount_amount</code>	Remise en pourcentage ou en valeur (centimes)
<code>status, used_at</code>	Suivi de l'utilisation effective du coupon (base du calcul du taux de conversion, Chapter 3)
<code>expires_at</code>	Date d'expiration

conversion présentée dans le backoffice (Chapter 4) est en conséquence dérivée d'un signal de *demande* (recherches et messages du chat), et non d'un signal de *vente*.

TABLE B.5 – Table `products`

Colonne	Description
<code>tenant_id</code> , <code>external_id</code>	Identifiant du produit côté PrestaShop
<code>name</code> , <code>description</code> , <code>category_name</code> , <code>manufacturer_name</code>	Attributs indexés pour la recherche vectorielle (RAG)
<code>price</code> , <code>quantity</code>	Prix et stock, synchronisés depuis PrestaShop
<code>active</code> , <code>available_for_order</code>	Disponibilité du produit